Guía de Actividades Práctico-Experimentales Nro. 004

1. Datos Generales

Asignatura	Sistemas Digitales
Ciclo	5to
Unidad	1 – Microcontroladores para sistemas embebidos
Resultado de aprendizaje de la unidad	R1. Diseña arquitectura de microcontroladores, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	4
Título de la Práctica	Uso de interrupciones en un sistema de alerta doméstica
Nombre del Docente	Luis Darío Sinche C.
Fecha	6 de noviembre de 2026
Horario	Viernes 11h30 a 13h30
Lugar	Laboratorio de computación (simulación en Wokwi y prueba en la placa ESP32 del grupo)
Tiempo planificado en el Sílabo	3 horas (APE, semana 4): práctica presencial de menos de 1 h 30 min; el informe se termina fuera de clase
Modalidad de trabajo	Se trabaja y se entrega en parejas; todos simulan en Wokwi y la prueba física se hace con la placa del grupo, si la tiene
Placa	ESP32 DevKit V1 de 30 pines (módulo ESP32-WROOM-32)

2. Objetivo(s) de la Práctica:

Objetivo. Construir un sensor de puerta con un pulsador y un LED que detecte la apertura mediante una interrupción externa del ESP32, y comprobar experimentalmente por qué la interrupción no pierde eventos que el sondeo sí pierde y por qué hace falta el antirrebote.

Al terminar, la pareja podrá:

- Configurar una interrupción externa con `attachInterrupt()` y el flanco correcto.
- Escribir una ISR correcta: corta, con `IRAM_ATTR`, variables `volatile` y patrón bandera.

- Explicar con sus propias mediciones la diferencia entre sondeo e interrupción y el efecto del rebote.

3. Fundamento teórico

Sondeo e interrupción externa. Con sondeo (*polling*), el `loop()` pregunta una y otra vez por el estado del pin; si está ocupado (por ejemplo, en un `delay()`), un toque corto se pierde. Con una interrupción externa, el periférico GPIO vigila el pin por hardware y, cuando detecta el cambio, la CPU pausa el programa, ejecuta la ISR y vuelve al mismo punto. Por eso el evento no se pierde aunque el `loop()` esté ocupado.

Flancos con INPUT_PULLUP. Con la resistencia pull-up interna, el pin lee 1 con el pulsador suelto y 0 al presionarlo. Un flanco es el instante del cambio: `FALLING` (de 1 a 0) ocurre al presionar, `RISING` (de 0 a 1) al soltar y `CHANGE` en ambos casos. En esta práctica se usa `FALLING`.

attachInterrupt(). `attachInterrupt(digitalPinToInterrupt(21), isrPuerta, FALLING);` significa «cuando el pin 21 caiga de 1 a 0, llama a isrPuerta». Se escribe una sola vez en `setup()`, después de `pinMode(21, INPUT_PULLUP)`.

ISR, IRAM_ATTR, volatile y patrón bandera. La ISR es la función que llama el hardware; debe ser corta y no usar `Serial.print` ni `delay` (si tarda demasiado, el watchdog reinicia el ESP32). `IRAM_ATTR` la guarda en la RAM interna rápida para que esté siempre disponible. Toda variable que usen la ISR y el `loop()` debe ser `volatile`, para que el `loop()` la lea siempre de la memoria y no de una copia vieja. Con el patrón bandera, la ISR solo levanta una bandera y el `loop()` la baja y hace el trabajo lento.

Rebote y antirrebote. Un pulsador mecánico produce varios cambios rapidísimos en un solo toque, y con interrupción cada uno dispara la ISR. El antirrebote dentro de la ISR ignora los flancos que llegan menos de `T_REBOTE` milisegundos después del último aceptado, usando `millis()`.

4. Materiales y reactivos:

Hardware (por grupo, solo para la Parte C):

Cantidad	Componente
1	ESP32 DevKit V1 de 30 pines + cable USB de datos (no solo de carga)
1	Protoboard
1	Pulsador (botón) de 4 patas
1	LED rojo de 5 mm
1	Resistencia de 330 Ω
Varios	Cables jumper macho-macho

En Wokwi (todos) se usan el ESP32 DevKit V1, 1 pulsador, 1 LED rojo y 1 resistencia de 330 Ω; el circuito completo ya viene en el diagram.json del Anexo A.

5. Equipos y herramientas

Software (cada estudiante):

- Wokwi (wokwi.com), en el navegador, sin instalar nada.
- Arduino IDE con el paquete esp32 by Espressif Systems versión 3.x, sin librerías externas, para la Parte C.
- Cuenta de GitHub y acceso al EVA, para la Parte D.

6. Procedimiento / Metodología

Práctica en parejas. El pulsador hace de sensor de puerta (presionado = puerta abierta), el LED es la luz de alarma y el Monitor Serie hace de notificación. Primero se comprueba en Wokwi que el sondeo pierde aperturas y que la interrupción no; luego se mide el efecto del rebote y, por último, se prueba en la placa real. **Antes de la clase:** lean el Fundamento teórico.

Tabla de conexiones

Todos los pines elegidos son seguros: no son de arranque o strapping (0, 2, 5, 12, 15), ni de la memoria flash (6–11), ni solo de entrada (34–39, que además no tienen pull-up interna).

Elemento	Pin del ESP32	Rótulo en la placa	Conexión
Pulsador (sensor de puerta)	GPIO21	D21	Una pata del pulsador → D21; la pata opuesta en diagonal → GND. Se usa la pull-up interna: suelto = 1, presionado = 0
LED (luz de alarma)	GPIO19	D19	GPIO19 → resistencia 330 Ω → ánodo del LED (pata larga); cátodo (pata corta) → GND
Tierra	GND	GND	Pulsador y cátodo del LED

Cuidados: los pines del ESP32 trabajan a 3,3 V; no conecten 5 V a ningún GPIO. Usen patas del pulsador que estén en diagonal: las patas del mismo lado pueden estar unidas internamente.

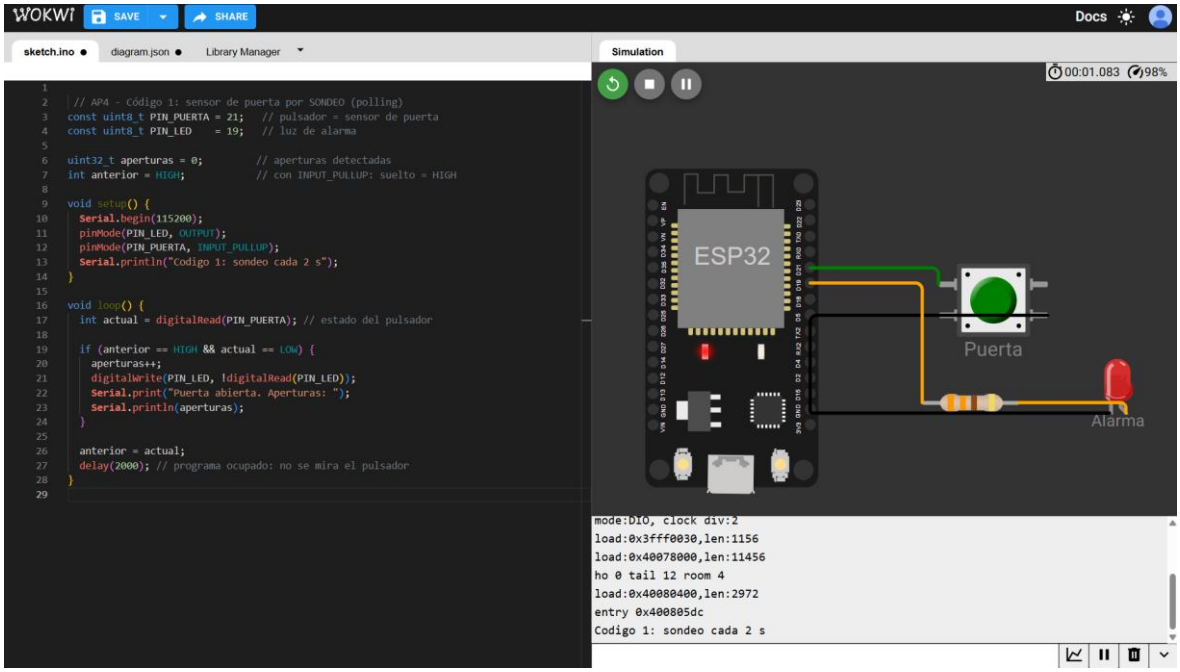
Parte A. El sondeo pierde aperturas (Wokwi)

El Código 1 revisa el pulsador por sondeo, una sola vez cada 2 segundos; el `delay(2000)` simula un programa ocupado.

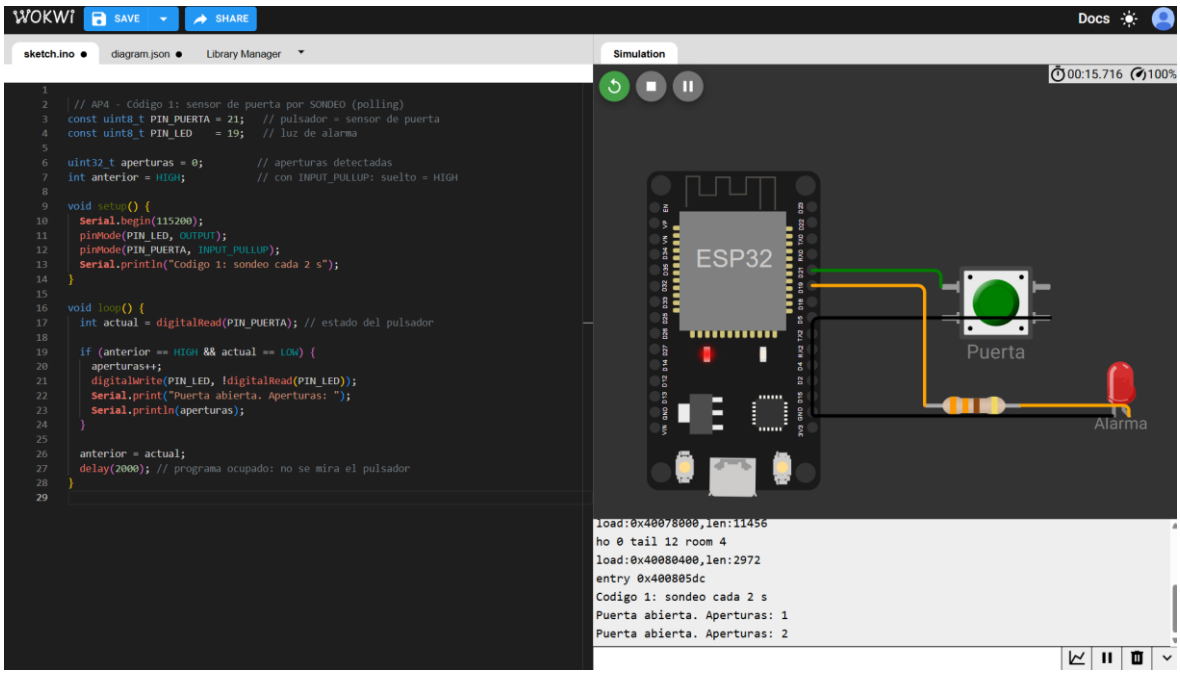
A1. Entren a wokwi.com, creen un proyecto nuevo de **ESP32** y abran la pestaña diagram.json.

A2. Borren todo su contenido y peguen el Anexo A. Deben aparecer el ESP32, el pulsador, la resistencia y el LED ya cableados.

A3. Abran la pestaña sketch.ino, borren su contenido y peguen el Código 1.



A4. Presionen ▸ (Start). Presionen el pulsador **10 veces**, con toques cortos y rápidos (unos 10 segundos en total).



A5. Anoten en la Tabla A cuántas aperturas reportó el Monitor Serie. Detengan la simulación y repitan una vez más.

A6. Tomen una captura de pantalla del Monitor Serie.

Código 1. Placa: ESP32 Dev Module · core Espressif 3.x · sin librerías · pines GPIO21 (pulsador) y GPIO19 (LED)

```
// AP4 - Código 1: sensor de puerta por SONDEO (polling)
const uint8_t PIN_PUERTA = 21;    // pulsador = sensor de puerta
const uint8_t PIN_LED    = 19;    // luz de alarma

uint32_t aperturas = 0;           // aperturas detectadas
int anterior = HIGH;              // con INPUT_PULLUP: suelto = HIGH

void setup() {
  Serial.begin(115200);
  pinMode(PIN_LED, OUTPUT);
  pinMode(PIN_PUERTA, INPUT_PULLUP);
  Serial.println("Codigo 1: sondeo cada 2 s");
}
```

```
void loop() {
  int actual = digitalRead(PIN_PUERTA);      // ¿cómo está el pulsador AHORA?
  if (anterior == HIGH && actual == LOW) {  // pasó de suelto a presionado
    aperturas++;
    digitalWrite(PIN_LED, !digitalRead(PIN_LED));
    Serial.print("Puerta abierta. Aperturas: ");
    Serial.println(aperturas);
  }
  anterior = actual;
  delay(2000);    // programa ocupado: aquí NO se mira el pulsador
}
```

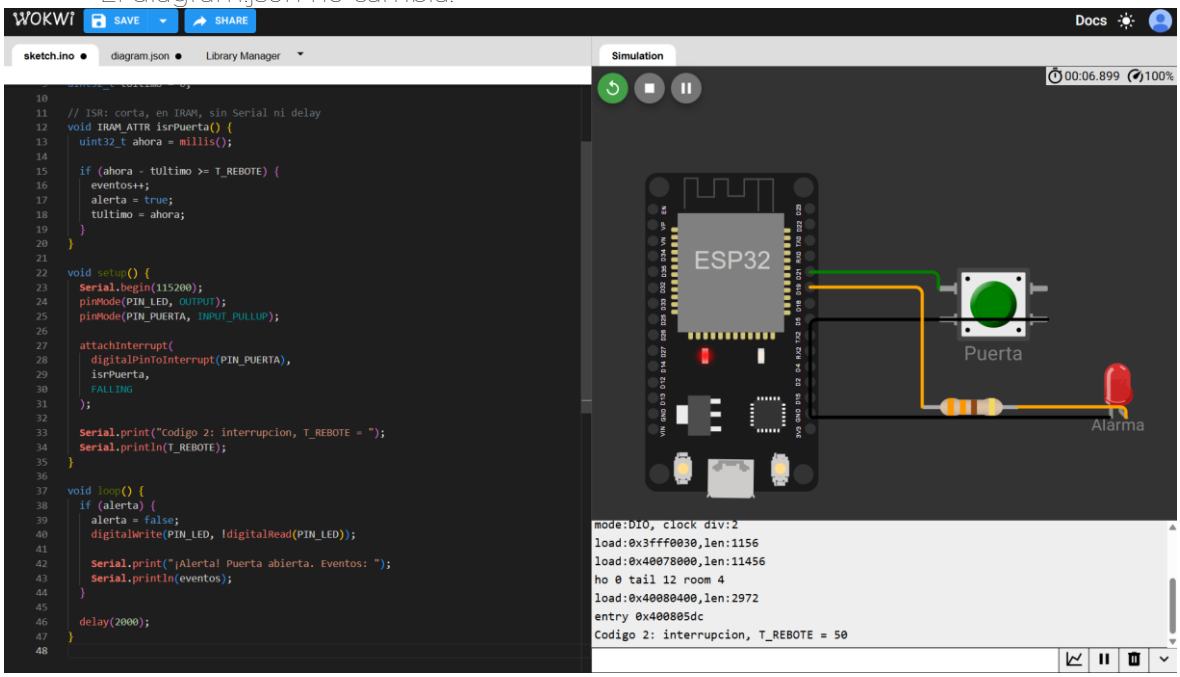
Tabla A. Sondeo: aperturas reportadas con 10 pulsaciones

Intento	Pulsaciones realizadas	Aperturas reportadas
1	10	2
2	10	0

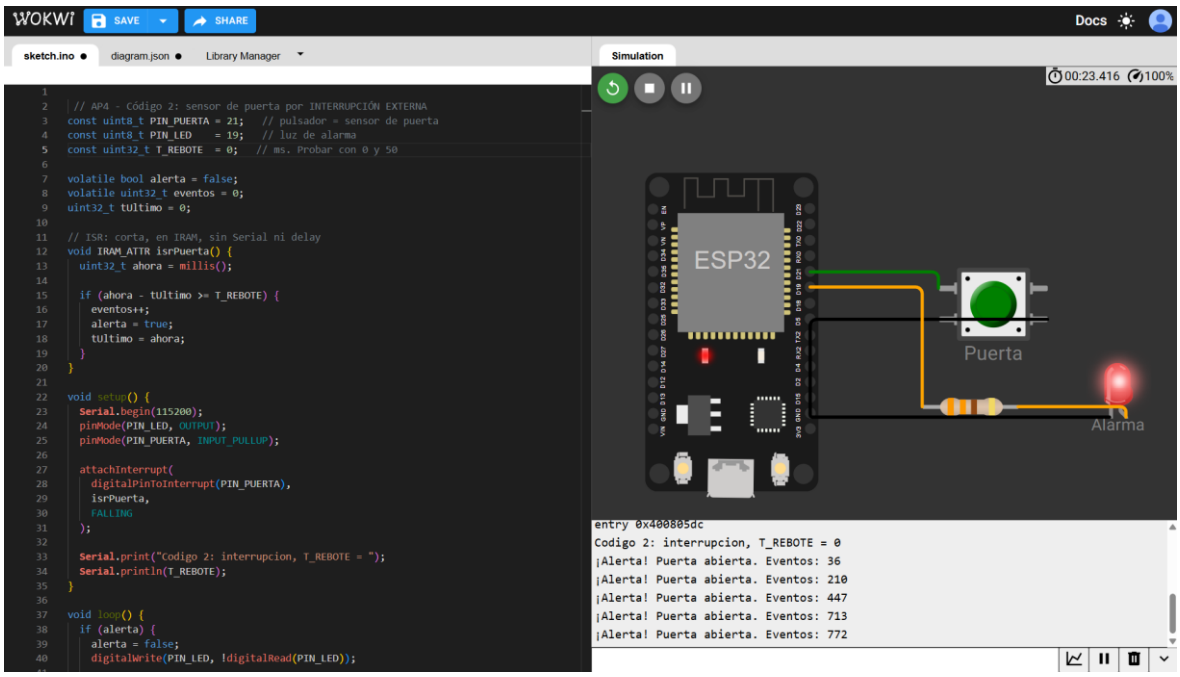
Parte B. La interrupción no pierde aperturas, pero el rebote estorba (Wokwi)

El Código 2 detecta la apertura con una interrupción `FALLING`. La ISR aplica el antirrebote, suma 1 al contador `eventos` y levanta la bandera; el `loop()` la baja, cambia el LED e imprime. El `delay(2000)` se deja a propósito: el programa sigue ocupado, pero ahora el pin lo vigila el hardware. El pulsador de Wokwi simula el rebote.

B1. En el mismo proyecto, reemplacen el contenido de sketch.ino por el Código 2. El diagram.json no cambia.

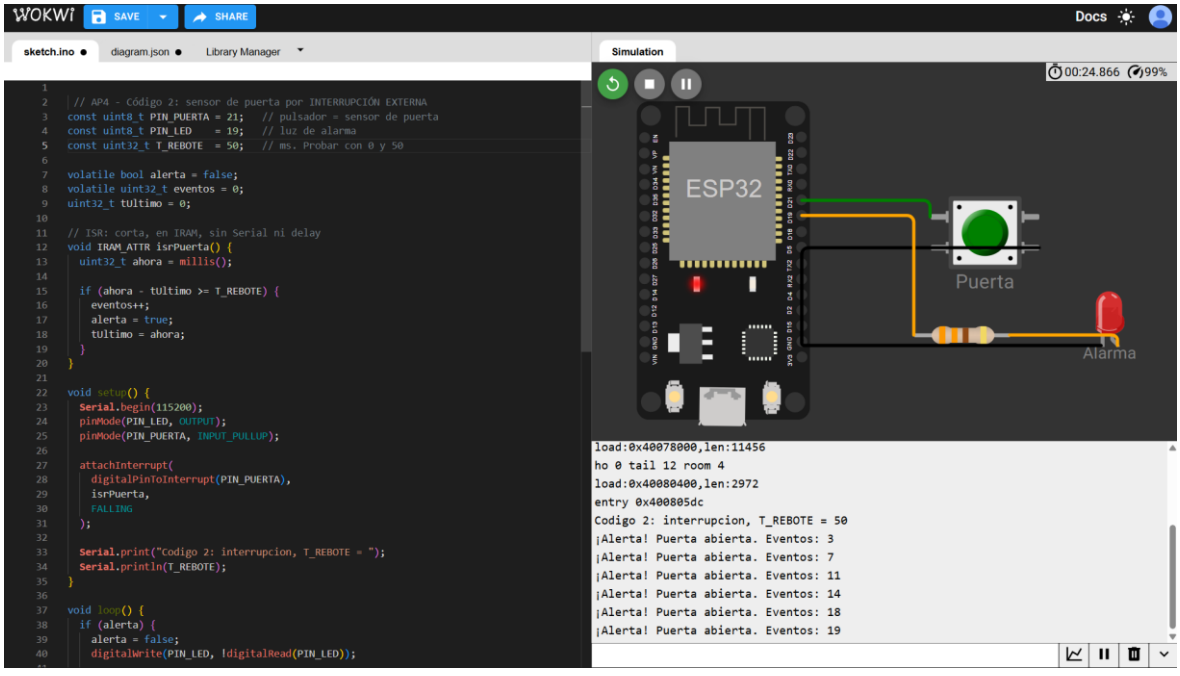


B2. Cambien la constante a `T_REBOTE = 0` (antirrebote desactivado). Presionen ▶ y presionen el pulsador **10 veces**.



B3. Anoten en la Tabla B el último valor de **Eventos** que muestra el Monitor Serie. Detengan y repitan una vez más.

B4. Vuelvan a `T_REBOTE = 50` y repitan los pasos B2 y B3.



B5. Tomen una captura del Monitor Serie para cada valor de `T_REBOTE`, guarden el proyecto en Wokwi y copien su enlace.

Código 2. Placa: ESP32 Dev Module · core Espressif 3.x · sin librerías · pines GPIO21 (pulsador) y GPIO19 (LED)

```
// AP4 - Código 2: sensor de puerta por INTERRUPCIÓN EXTERNA
const uint8_t  PIN_PUERTA = 21;    // pulsador = sensor de puerta
const uint8_t  PIN_LED    = 19;    // luz de alarma
const uint32_t T_REBOTE   = 50;    // ms. Parte B: probar con 0 y con 50

volatile bool     alerta  = false; // bandera: la usan la ISR y el loop
volatile uint32_t eventos = 0;     // contador: lo usan la ISR y el loop
uint32_t tUltimo = 0;              // solo lo usa la ISR

// ISR: corta, en IRAM, sin Serial ni delay
void IRAM_ATTR isrPuerta() {
  uint32_t ahora = millis();
  if (ahora - tUltimo >= T_REBOTE) {  // antirrebote
    eventos++;
    alerta  = true;                   // levanta la bandera
    tUltimo = ahora;
  }
}
```

```
void setup() {
  Serial.begin(115200);
  pinMode(PIN_LED, OUTPUT);
  pinMode(PIN_PUERTA, INPUT_PULLUP);  // 1.° el pin
  attachInterrupt(digitalPinToInterrupt(PIN_PUERTA),
                  isrPuerta, FALLING);   // 2.° la interrupción
  Serial.print("Codigo 2: interrupcion, T_REBOTE = ");
  Serial.println(T_REBOTE);
}

void loop() {
  if (alerta) {                        // ¿está levantada la bandera?
    alerta = false;                    // bajarla
    digitalWrite(PIN_LED, !digitalRead(PIN_LED));
    Serial.print("¡Alerta! Puerta abierta. Eventos: ");
    Serial.println(eventos);           // el trabajo lento va en el loop
  }
  delay(2000);    // programa ocupado: el pulsador lo vigila el hardware
}
```

El LED y el mensaje pueden aparecer hasta 2 segundos después del toque, por el `delay(2000)`; lo que importa es que el contador `eventos` no pierde ninguna apertura.

Tabla B. Interrupción en Wokwi: eventos contados con 10 pulsaciones

T_REBOTE (ms)	Eventos, intento 1	Eventos, intento 2	¿Más, menos o igual a 10?
0	772	0	Mayor
50	19	0	maroy

Parte C. Prueba en la placa ESP32 real (grupos con placa)

Los grupos sin placa no realizan esta parte y no pierden puntos (ver Evaluación); en su lugar adelantan las preguntas de control.

C1. Con el USB desconectado, armen en la protoboard el pulsador y el LED según la Tabla de conexiones.

C2. Conecten el cable USB. En Arduino IDE elijan Herramientas → Placa → ESP32 Dev Module y el puerto correcto. Carguen el Código 2 con `T_REBOTE = 0`.

C3. Abran el Monitor Serie a 115200 baudios. Si no aparece el mensaje inicial, presionen el botón EN de la placa.

C4. Presionen el pulsador 10 veces y anoten los eventos en la Tabla C. Repitan con `T_REBOTE = 50`.

C5. Tomen una foto del montaje funcionando.

Tabla C. Placa real: eventos contados con 10 pulsaciones

T_REBOTE (ms)	Eventos, intento 1	Eventos, intento 2	Observaciones
0			
50			

Si algo falla, revisen el Anexo B (Solución de problemas).

Parte D. Entrega

D1. Suban al repositorio de GitHub del grupo una carpeta AP4 con:

- los dos códigos (.ino);
- el diagram.json;
- las capturas del Monitor Serie y, si hubo Parte C, la foto del montaje.

D2. Suban al EVA, en la tarea de la AP4, el informe en PDF con el enlace al repositorio y el enlace al proyecto de Wokwi.

D3. Plazo: el que indique el docente en el EVA.

7. Resultados esperados:

- **Código 1 (sondeo):** con 10 pulsaciones rápidas, el Monitor Serie reporta menos de 10 aperturas (Tabla A).
- **Código 2 con T_REBOTE = 0:** el contador `eventos` suele superar 10, porque cada rebote dispara la ISR (Tabla B).
- **Código 2 con T_REBOTE = 50:** el contador marca 10 o un valor muy cercano: no se pierden aperturas ni se cuentan rebotes.
- **Placa real (Parte C):** el mismo comportamiento que en Wokwi; el número de rebotes depende de cada pulsador (Tabla C).

Se debe entregar el esquema del circuito, los dos códigos, las capturas y las Tablas A, B y C completas.

8. Preguntas de Control:

Respondan en el informe, con sus palabras.

1. Con el Código 1, ¿por qué se perdieron aperturas? Explíquenlo con lo que hace el `loop()` durante el `delay(2000)`.

Porque el loop() revisa el pulsador y luego se detiene dos segundos en delay(2000). Si la puerta se abre y cierra durante esa espera, el programa no vuelve a leer el estado del pin y no cuenta ese toque.

2. El Código 2 también tiene un `delay(2000)`. ¿Por qué esta vez no se pierden aperturas? ¿Quién vigila el pin mientras el `loop()` espera?

Porque la interrupción externa vigila los cambios del GPIO21 por hardware. Cuando detecta el flanco FALLING, ejecuta la ISR, incrementa eventos y levanta la bandera alerta aunque el loop() esté ocupado. Después, el loop() muestra el aviso.

3. ¿Por qué el contador marcó más de 10 eventos con `T_REBOTE = 0`? ¿Qué cambió con 50 ms?

El pulsador puede producir varios cambios eléctricos durante una misma pulsación, lo que se conoce como rebote. Con T_REBOTE = 0 no se descartan esos cambios. Al aplicar 50 ms, la ISR ignora los flancos cercanos al último aceptado y se reducen los registros adicionales. En las capturas se observan 772 y 19 eventos, respectivamente, aunque no consta el número exacto de toques de cada prueba.

4. En el Código 2, ¿por qué `alerta` y `eventos` son `volatile` y `tUltimo` no?

Las variables alerta y eventos las comparten la ISR y el loop(), por eso se declaran volatile para evitar que el compilador asuma que permanecen constantes. En cambio, tUltimo solo es utilizada dentro de la ISR y en este código no hace falta declararla volatile.

5. ¿Por qué el `Serial.print` está en el `loop()` y no en la ISR? ¿Qué podría pasar si se mueve a la ISR?

Porque imprimir por el puerto serie toma tiempo y la interrupción debe ser lo más corta posible. Si la ISR imprime o usa delay(), puede bloquear otras tareas e incluso provocar reinicios del ESP32. Por eso solo actualiza la bandera y el contador.

6. Si se usara `RISING` en lugar de `FALLING`, ¿en qué momento se encendería la alarma? ¿Por qué?

Como INPUT_PULLUP mantiene el pin en HIGH cuando el botón está suelto, al presionar pasa a LOW y se produce FALLING. RISING ocurre al soltarlo, cuando vuelve de LOW a HIGH. En consecuencia, el aviso se activaría al soltar el pulsador, no al presionarlo.

9. Entregables y estructura del informe

El informe debe tener la siguiente estructura:

- Portada con los datos de la UNL, los nombres de la pareja, el paralelo, la fecha y los enlaces al repositorio y a Wokwi.
- Objetivo.
- Esquema del circuito: captura del diagrama de Wokwi y, si aplica, foto del montaje real.
- Resultados: Tablas A, B y C completas, con las capturas del Monitor Serie.
- Respuestas a las preguntas de control.
- Conclusiones: tres como máximo, basadas en sus mediciones.
- Declaración de uso de IA generativa: qué herramienta usaron, para qué y qué parte del informe se vio afectada; si no usaron ninguna, declárenlo.

Entrega: la carpeta AP4 en el repositorio de GitHub del grupo (Parte D) y, en la tarea de la AP4 del EVA, el informe en PDF con los enlaces.

Declaracion de IA:
Para el desarrollo de las preguntas de control de la presente práctica, se utilizó la herramienta de inteligencia artificial ChatGPT como apoyo para comprender el funcionamiento de las interrupciones externas, el antirrebote y las variables utilizadas en el ESP32. Esta herramienta permitió orientar y mejorar la redacción de las respuestas, tomando como referencia los contenidos de la guía práctica. Las respuestas fueron revisadas con la finalidad de mantener coherencia con los temas estudiados y las actividades realizadas.

10. Evaluación

Calificación sobre 10 puntos. Cada nivel vale el porcentaje indicado de los puntos del criterio. Los grupos sin placa no pierden puntos por la Parte C: esa parte no se califica y la nota se calcula sobre 9 puntos y se lleva a 10 (nota × 10 / 9).

Criterio	Puntos	Excelente (100 %)	Satisfactorio (70 %)	En desarrollo (40 %)	No presenta (0 %)
Parte A: Tabla A y captura	1,5	Tabla completa, con captura y dos intentos	Tabla completa sin captura	Un solo intento o datos incoherentes	No presenta
Parte B: Tabla B y capturas	2,5	Tabla completa con los dos valores de T_REBOTE y capturas	Tabla completa; falta una captura	Solo un valor de T_REBOTE medido	Sin mediciones ni capturas
Parte C: prueba en placa real	1	Montaje funcionando, Tabla C completa y foto	Montaje funcionando; Tabla C incompleta	Montaje con fallas o sin evidencia	Sin evidencia
Preguntas de control	3	Las 6 respuestas correctas y bien justificadas	4 o 5 correctas	2 o 3 correctas	Menos de 2
Repositorio de GitHub	1	Carpeta AP4 con todos los archivos pedidos	Falta un archivo	Faltan varios archivos	Sin repositorio
Informe y declaración de uso de IA	1	Estructura completa, conclusiones basadas en datos y declaración de IA	Conclusiones genéricas o falta un apartado	Faltan varios apartados	Falta la declaración o la estructura

La retroalimentación se entregará en un máximo de 5 días hábiles.

11. Bibliografía

1. Freire Sánchez, L. A. (2023). Electrónica y sistemas embebidos: una visión a nivel técnico y tecnológico. ISBN 978-9942-636-22-5.
2. Zailani, N. F. (2024). Getting started with Arduino and ESP32: A beginner's guide with Wokwi. Politeknik Mersing.
3. Espressif Systems. (s. f.). ESP32 Series Datasheet. https://www.espressif.com/en/support/documents/technical-documents
4. Espressif Systems. (s. f.). Arduino-ESP32 documentation. https://docs.espressif.com/projects/arduino-esp32/
5. CodeMagic LTD. (2025). Wokwi – documentación. https://docs.wokwi.com
6. Arduino. (2024). Arduino Language Reference. https://www.arduino.cc/reference/

12. Elaboración y Aprobación

Elaborado por	ING. LUIS DARÍO SINCHE CUEVA **DOCENTE**	Validar únicamente en FirmaEC. Firmado electrónicamente por: **LUIS DARIO SINCHE CUEVA**
Revisado por	ING. RAMIRO RENE RIVERA GUAMÁN **TÉCNICO DOCENTE**	Validar únicamente en FirmaEC. Firmado electrónicamente por: **RAMIRO RENE RIVERA GUAMAN**
Aprobado por	ING. EDISON LEONARDO CORONEL ROMERO **DIRECTOR DE CARRERA**	**Firmado electrónicamente por:** **EDISON LEONARDO CORONEL ROMER** **Validar únicamente con FirmaEC**

Anexo A. diagram.json de Wokwi

Peguen este contenido en la pestaña diagram.json (paso A2).

```
{
  "version": 1,
  "author": "Sistemas Digitales UNL",
  "editor": "wokwi",
  "parts": [
    { "type": "wokwi-esp32-devkit-v1", "id": "esp", "top": 0, "left": 0, "attrs": {}
},
    { "type": "wokwi-pushbutton", "id": "btn1", "top": 60, "left": 180,
      "attrs": { "color": "green", "label": "Puerta" } },
    { "type": "wokwi-resistor", "id": "r1", "top": 140, "left": 170,
      "attrs": { "value": "330" } },
    { "type": "wokwi-led", "id": "led1", "top": 110, "left": 270,
      "attrs": { "color": "red", "label": "Alarma" } }
  ],
  "connections": [
    [ "esp:TX0", "$serialMonitor:RX", "", [] ],
    [ "esp:RX0", "$serialMonitor:TX", "", [] ],
    [ "esp:D21", "btn1:1.l", "green", [] ],
    [ "btn1:2.r", "esp:GND.1", "black", [] ],
    [ "esp:D19", "r1:1", "orange", [] ],
    [ "r1:2", "led1:A", "orange", [] ],
    [ "led1:C", "esp:GND.1", "black", [] ]
  ]
}
```

Si las piezas aparecen encimadas, arrástrenlas con el mouse: la posición no cambia las conexiones.

Anexo B. Solución de problemas

Antes de pedir ayuda, revisen esta tabla.

Síntoma	Causa probable	Qué hacer
En el Arduino IDE no aparece «ESP32 Dev Module»	No está instalado el core del ESP32	Herramientas → Placa → Gestor de placas → buscar «esp32» → instalar «esp32 by Espressif Systems» (versión 3.x)
Error de compilación «stray \302» o parecido	Al copiar desde Word se pegaron caracteres invisibles o comillas tipográficas	Borren y escriban a mano la línea que marca el error; usen comillas rectas (")
El Monitor Serie muestra caracteres raros	Velocidad distinta de la del código	Pongan el Monitor Serie en 115200 baudios
No aparece ningún mensaje en la placa	El mensaje inicial salió antes de abrir el monitor	Presionen el botón EN de la placa
No se puede cargar el programa («Failed to connect»)	Puerto incorrecto o cable solo de carga	Elijan el puerto correcto o cambien de cable; si persiste, mantengan presionado BOOT al empezar la carga
La alarma no responde nunca	Patas del pulsador unidas internamente o pin equivocado	Usen patas en diagonal; revisen que vaya a D21 y a GND
Aparecen alarmas sin tocar nada	Pin flotando: falta INPUT_PULLUP, se usó un pin 34–39 o hay un cable flojo	Revisen pinMode(PIN*PUERTA*, *INPUT*PULLUP), usen D21 y ajusten los cables
La alarma se activa al soltar, no al presionar	Se usó RISING	Usen FALLING

Síntoma	Causa probable	Qué hacer
El ESP32 se reinicia y aparece «Guru Meditation Error»	Se puso Serial.print o delay dentro de la ISR	Dejen la ISR como en el Código 2; el trabajo lento va en el loop()
El LED no enciende	LED invertido o resistencia en otra fila de la protoboard	Pata larga hacia la resistencia y D19; pata corta a GND
El LED no enciende en Wokwi	El diagram.json no se pegó completo	Borren todo el contenido de diagram.json y peguen de nuevo el Anexo A

Enlace Git: https://github.com/Orojco/Orozco-Marco.git